

最终赛题报告：nt-devskill

赛题： T3-1-1 NineToothed 算子开发 Skill 小组：新建小组名 Skill 名称： nt-devskill 版本： 1.8 提交日期： 2026-07-12

摘要

nt-devskill 是一套面向 **NineToothed DSL** 的工程化 .skill 包，把"算子需求 → 可上线九齿 kernel"的全流程封装成可被主流 AI 编程智能体调用的强制工作流。本次提交同时附带一个独立的配套 MCP Server (remote-operator-mcp)，为 Step 0 → Step 3 的远程 GPU 闭环提供 12 个标准化工具 (upload_code / run_test / run_python / auto_bench / mx_smi / remote_ls / remote_cat / remote_md5 / remote_glob / remote_rm / download_file / ping)，可一键注册到 Qoder / Cursor / Claude Code / Windsurf / Cline / Codex CLI 等主流 Agent。本报告覆盖：设计原则与包结构、核心工作流、5 个自测算子任务 (覆盖 elementwise / reduction / composition / scatter / layout-view 5 族)、benchmark 设计与性能回退分析、失败诊断案例 (含 Step 0.8 方案先行确认机制的正反面验证)、与无 Skill 基线的 A/B 对照、安全/依赖/授权/引用披露 (含 MCP 的安全与凭据管理)，以及后续可维护计划。

1. .skill 目标、设计原则与包结构

1.1 目标

把九齿算子开发中"隐性工程知识"显性化为 Agent 可读的决策树，使任意合格的 AI 编程智能体在拿到一个新算子需求时能：

- 1. 在 1 轮对话内输出分类结论与实现方案 (而非靠试错)
- 2. 在动笔写代码前先向用户披露方案与风险 (方案先行确认)
- 3. 复用 12 个内置示例算子的 arrangement / application 模板
- 4. 在测试失败时根据 25 张 FIX_CARDS 进行症状优先的诊断
- 5. 在性能不达标时按 Roofline 模型做 tile sweep + overhead breakdown

1.2 设计原则

原则	体现
可被任意 Agent 调用	纯 Markdown 工作流 + 只读参考目录；不绑定特定 IDE
只读 Skill 目录	examples/、scripts/、references/ 均为参考模板；新算子必须写入用户工程目录
方案先行	Step 0.5 (能力判定) + Step 0.8 (📄 IMPLEMENTATION PROPOSAL 简报)
闭环验证	Step 2.5 强制 pytest 闭环；Step 2.8 强制 benchmark + 优化 + 复测
可复现	所有命令、环境、日志路径均在自测报告中披露
反 reward hacking	inspect_generated.py 检查全零/无操作/常数 store；atomic RMW 路径审查
远程闭环标准化	配套独立 MCP Server (remote-operator-mcp)，12 个工具覆盖上传/测试/监控/清理，可注册到任意主流 Agent 的 stdio MCP 配置

1.3 包结构

```
competition/                                # 本次竞赛提交根目录
├── nt-devskill/                             # Skill 主体（只读，可独立分发）
│   ├── SKILL.md                             # 914 行工作流主文件
│   ├── README.md                           # 安装与使用（含 Qoder/Cursor/Claude/Cline/Copilot 等 Agent 安装指引）
│   └── examples/                            # 12 个示例算子（只读）
│       ├── add/ silu/ softmax/ matmul/ bmm/ addmm/ sdpa/
│       └── fused_rms_norm/ swiglu/ conv2d/ rope/ max_pool2d/
│   ├── scripts/                             # pipeline / validate / benchmark / doctor / diag_*
│   ├── references/
│       ├── API_REFERENCE.md                 # NineToothed + libdevice API 速查
│       ├── CODE_TEMPLATES.md               # 15 种 arrangement 模板
│       ├── TAXONOMY.md                     # 9 族分类路由
│       ├── FIX_CARDS.md                    # 25 张故障诊断卡
│       ├── LAYOUT.md                       # 非连续/stride/offset 参考
│       └── OPTIMIZATION_GUIDE.md           # Roofline + tile sweep + overhead
│   ├── specs/                              # 12 个算子 YAML 规格卡
│   ├── tests/                              # pytest 正确性 + 性能基准
│   └── agents/                             # Explore / Plan 子代理配置
├── remote-operator-mcp/                     # 配套 MCP Server（独立、与 Skill 解耦）
│   ├── README.md                           # 12 个工具说明 + 多 Agent 安装指引
│   ├── pyproject.toml                      # Python 3.12 + mcp>=1.27.2 + paramiko>=5.0.0
│   ├── uv.lock
│   ├── config.example.json                 # SSH 连接模板（不含真实凭据）
│   └── remote_operator_mcp.py              # MCP 入口
├── 测试场地/                               # 自测工程（5 个算子 + copysign A/B 对照）
├── 新建队伍名_九齿skill创新挑战_T3-1-1_赛题报告.pdf # 本最终报告
├── 董俊宏_九齿skill创新挑战_proposal.pdf      # proposal
├── PR_DESCRIPTION_TEMPLATE.md                # PR 描述模板
├── REFERENCE.md                             # 公开引用 + AI 辅助披露
└── HONOR_CODE.md                           # 竞赛合规声明
```

注：新建队伍名_九齿skill创新挑战_T3-1-1_赛题报告.pdf / PR_DESCRIPTION_TEMPLATE / REFERENCE / HONOR_CODE 属本次提交的**提交级文档**，不是 Skill 包的一部分；Skill 包自身只需 SKILL.md + README.md + examples/ + scripts/ + references/ + specs/ + tests/ + agents/ 即可独立运行。

2. 核心工作流说明

Skill 工作流由 9 个 Step 组成，其中 Step 0/0.5/0.8/2.5/2.8 为强制步骤：

Step	名称	强制	作用
0	分析工程与测试环境	✔	探测 GPU/MCP/SSH，选定测试执行策略
0.5	能力可行性判定	✔	九齿不能表达时输出 CAPABILITY REPORT + A/B/C 方案
0.8	实现方案确认	✔	输出 📄 IMPLEMENTATION PROPOSAL 简报等待用户确认
1	任务分类与路由	✔	按 TAXONOMY.md 归入 9 族之一
2	代码生成	✔	arrangement + application + wrapper + test 四件套
2.5	测试闭环	✔	Write → Test → Fix，最多 3 轮修复、10 次迭代
2.8	性能优化	✔	Diagnose → Sweep → Optimize → Verify（4 轮）
3	审计笔记	✔	写入自测结果报告（环境/命令/日志/精度指标）

关键机制：

- **方案先行确认 (Step 0.8)**：任何写代码前必须先输出 5 项简报（分类结论、倾向方案、是否 GPU kernel、潜在风险、备选 A/B/C），用户回复"直接开始 / 换 B / 换 C / 提出调整"才能继续
- **能力判定 (Step 0.5)**：九齿不能表达（如纯 view-only 主操作、需要动态 shape、需要 CUDA Graph）时输出 CAPABILITY REPORT 并给 A/B/C 方案，把选择权交给用户
- **Stop rules**：单算子最多 3 轮修复、10 次迭代；超出则分类为"失败诊断案例"并写入报告

3. 自测算子任务、运行过程与 correctness 结果

5 个自测任务覆盖 5 个不同算子族，超出赛题"至少 4 个自测任务、每个 2.5 分"的最低要求，并且第 5 个任务（pixel_unshuffle）专门用来验证 Step 0.8 方案先行确认机制的有效性。

3.1 任务总览

#	算子	算子族	工程目录	自测报告
1	copysign	elementwise (含 libdevice)	测试场地/ntops11	自测结果_copysign_with_skill.md
2	nansum	reduction	测试场地/ntops7	自测结果_nansum_with_skill.md
3	addcmul	composition (诊断/修复)	测试场地/ntops9	自测结果_addcmul_diag_and_fix.md
4	scatter_add	scatter (atomic RMW)	测试场地/ntops13	自测结果_scatter_add_with_skill.md
5	pixel_unshuffle	layout/view (重排)	测试场地/ntops14	自测结果_pixel_unshuffle_with_skill.md

3.2 copysign (elementwise)

- **提示词**：实现一个九齿算子 copysign(input, other)，要求支持 fp32/fp16 与 IEEE 754 ±0.0
- **分类**：ElementWise（双输入），复用 ntops.kernels.element_wise.arrangement
- **关键设计**：分 dtype 双路径
 - fp16: bitcast → abs_bits = in & 0x7FFF + sign_bits = ot & 0x8000 → bitcast 回 fp16
 - fp32: libdevice.copysign (API_REFERENCE.md 明确禁止 < 0.0，绕过 signed-zero 陷阱)
- **Correctness**：首轮 12/12 PASS in 14.19 s；优化后 12/12 PASS in 8.57 s
- **覆盖**：shape×dtype 参数化 8 项 + IEEE 754 ±0.0 专项 2 项 + 特殊值 2 项

3.3 nansum (reduction)

- **提示词**：实现一个九齿算子 nansum(input, dim, keepdim)，NaN 视为 0
- **分类**：Reduction，参考 softmax / fused_rms_norm 模板
- **关键设计**：NaN_mask = (x != x) (API_REFERENCE.md 推荐，避免走 isnan() 库调用)
- **Correctness**：
 - R1 (buggy)：Triton 编译失败，触发 FC-17 类 (dtype attribute path)
 - R2 (修复)：46/46 PASS in 14.19 s
 - 覆盖: shape (128,256)/(64,8192)/(1024,1024)/(4,32,128) × dtype fp32/fp16 × 非连续输入 × keepdim × 全 NaN 行
- **失败诊断**：R1 失败后根据生成 IR 反推 FC-17 类 fix，一轮修复 PASS

3.4 addcmul (composition / 诊断修复)

- **提示词**：诊断 ntops.kernels.addcmul 的 fp16 精度 bug 并最小修复
- **分类**：Composition (input + value * tensor1 * tensor2)
- **根因**：FC-15 (fp16 精度不足，累加/混合运算未 upcast)
 - buggy 代码: prod = (t1.fp32 * t2.fp32).to(fp16) → output = input + v * prod 在 fp16 执行，误差随 |value| 线性放大
- **最小修复**：把 input / tensor1 / tensor2 / value 全部 upcast 到 fp32，整段表达式在 fp32 计算后再 cast 回原 dtype
- **Correctness**：
 - Before: basic fp32 PASS、fp16/large FAIL (MARE ≈ 9.7e-3 ~ 1.45e-2)
 - After: **4/4 PASS**, fp16 bitwise exact (max_diff = 0.0)
- **Benchmark**：Before → After, 4 组 timing 差异均在噪声范围内 (-7.7% ~ +8.3%)，**无性能回退**

3.5 scatter_add (scatter / atomic RMW)

- **提示词：** 实现一个九齿算子 `scatter_add(self, dim, index, src)`
- **分类：** Scatter (不在 TAXONOMY 9 族内置列表中，归入 "未知族" 并触发  IMPLEMENTATION PROPOSAL)
- **关键设计：**
 - Triton kernel 用 `tl.atomic_add`，生成 IR 中含 `tt.atomic_rmw fadd, acq_rel` (80/106 artifact 文件)
 - fp32 working buffer：无论 `self/src dtype`，kernel 总在 fp32 累加
- **Correctness：**
 - R1 (V1 kernel，直接 atomic on input dtype)：34/65 (fp32 PASS、fp16 全 FAIL，`max_diff` 1.2e+01 ~ 3.3)
 - R2 (V2 kernel，fp32 working buffer)：**65/65 PASS**
- **覆盖：** 1D/2D/3D/4D × fp32/fp16 × 多种 dim × high-conflict × all-zero-index

3.6 pixel_unshuffle (layout/view 重排)

- **提示词：** 实现一个九齿算子 `pixel_unshuffle(input, downscale_factor)`，等价 `torch.nn.functional.pixel_unshuffle`，把 (... , C, H*r, W*r) 重排成 (... , C*r*r, H, W)，支持 3D/4D、fp32/fp16
- **分类：** Layout/view (重排类)，命中 TAXONOMY.md §9 + LAYOUT.md `pixel_unshuffle` 指引
- **Step 0.8 方案简报 (5 项全部披露)：**
 - i. 分类结论：layout/view (重排)，数学本质 `reshape + permute + reshape`
 - ii. 倾向方案 (Plan A)：wrapper-heavy + Pattern 13 1D copy kernel
 - iii. GPU kernel 判断：**需要** (MetaX 上 `F.pixel_unshuffle` 已返回 `contiguous`，必须做真数据搬运，不能纯 view)
 - iv. 风险：MetaX 非连续 tensor flatten、大 `downscale_factor` memory access、fp16 精度 (不需要，纯数据搬运)
 - v. 备选：B 纯九齿 arrangement、C 纯 torch 回退
- **用户选 Plan A → 才动笔写代码**
- **关键设计：**
 - kernel: Pattern 13 1D copy (`def application(source, output): output = source`)
 - arrangement: `source.flatten().tile((block_size,)) + output.flatten().tile((block_size,))`
 - wrapper: 3D/4D 分支，`view + permute` 得到非连续中间 tensor，再交给 Pattern 13 kernel 做真数据搬运
- **测试闭环 (2 轮失败 → 第 3 轮 PASS)：**
 - R1 FAIL：误用 `element_wise.arrangement`，`ndim` 不匹配 (中间 5D vs 输出 4D)
 - R2 FAIL：`permute` 维度顺序错，`max_diff` = 4.085
 - R3 PASS：(1,1,4,4) 小 tensor 逐元素 `trace` 推导正确 `permute` → `view(N,C,H,r,W,r).permute(0,1,3,5,2,4)`
 - 最终：**12/12 PASS in 10.29s** (`block_size=4096`)
- **性能：** 小 shape 0.26–0.29x (MetaX GPU launch 最低延迟 ~0.04ms 框架上限)，**大 shape (8,64,256,256) 反超 torch：1.08x fp32 / 1.09x fp16** (带宽 144.9 vs 134.3 GB/s)
- **Skill 效能验证：** 本任务是 **Step 0.8 方案先行确认机制的正面验证** —— Agent 在动笔前输出完整 5 项简报等用户回复，全程没有"绕过九齿"或"silent fallback"，与 `ntops13/scatter_add` 绕过九齿事故形成对照

3.7 Correctness 汇总

算子	测试总数	PASS	FAIL	fp32	fp16	非连续	特殊场景
copysign	12	12	0	8/8	8/8	—	±0.0、inf、nan
nansum	46	46	0	23/23	23/23	4/4	全 NaN、keepdim
addcmul	4	4	0	2/2	2/2	—	value=3/10
scatter_add	65	65	0	全	全	—	high-conflict、all-zero
pixel_unshuffle	12	12	0	全	全	—	3D/4D × r=2/3/4、invalid_shape、invalid_ndim
合计	139	139	0				

4. Benchmark 设计、输入规模、性能结果与回退分析

4.1 Benchmark 设计

每个算子的 benchmark 覆盖：

- **输入规模矩阵**：1K → 64K → 1M → 16M 元素
- **Dtype**：fp32 / fp16
- **Timing**：CUDA events (GPU-only) + E2E (含 host overhead) + overhead breakdown
- **统计**：30 warmup + 200 iters，取 median
- **对照**：torch.<op> 在同机同时跑同一 timing 机制
- **指标**：speedup = torch_time / nt_time，MERE (mean relative error)，MARE (max absolute relative error)

4.2 性能结果 (MetaX C500)

算子	输入规模	dtype	nt (ms)	torch (ms)	speedup	瓶颈
copysign	(1024,)	fp32	—	—	0.18x	launch overhead
copysign	(1024,1024)	fp32	—	—	0.30x	memory-bound
copysign	(4096,4096)	fp32	—	—	0.99x ✓	memory-bound
copysign	(4096,4096)	fp16	—	—	0.98x ✓	memory-bound
nansum	(256,256)	fp32	0.038	0.011	0.28x	launch overhead
nansum	(1024,1024)	fp32	0.038	0.018	0.47x	Triton + NaN 指令
addcmul	(128,128)	fp32	0.12	—	—	memory-bound
addcmul	(4096,4096)	fp16	0.38	—	—	memory-bound
scatter_add	1D 1K	fp32	0.235	0.054	0.23x	kernel dispatch
scatter_add	2D 2048×512 high-conflict	fp32	0.370	0.098	0.26x	atomic 竞争
pixel_unshuffle	(2,3,64,64) r=2 4D	fp32	0.0561	0.0147	0.26x	launch overhead
pixel_unshuffle	(4,16,128,128) r=2 4D	fp32	0.0566	0.0382	0.68x	memory-bound
pixel_unshuffle	(8,64,256,256) r=2 4D	fp32	0.9261	0.9994	1.08x ✓	memory-bound (nt 反超)
pixel_unshuffle	(8,64,256,256) r=2 4D	fp16	0.9181	1.0001	1.09x ✓	memory-bound (nt 反超)

4.3 性能回退分析

回退 1: copysign 默认 block_size=1024 → 大 shape 0.50x

- **根因**：大 shape 下 tile=1024 使 grid 粒度过细，单 block 处理数据量不足
- **修复**：diag_tile_sweep.py 扫描 [256, 512, 1024, 2048, 4096, 8192]，最优 4096
- **修复后**：0.50x → 0.98x (fp16)，0.89x → 0.99x (fp32)

回退 2: nansum 在 (256,256) 上 0.28x

- **根因**：归约维度小 → kernel launch 占主导；host overhead 占比 ≈ 0% (非 host 问题)，GPU kernel 本身慢
- **诊断**：Triton 生成代码含 NaN 检测指令，相比 hand-optimized CUDA 多 2–3× 指令
- **规避**：属框架上限，无法通过 tile/warps 消除；在大 shape 上回落到 0.47x–0.52x

回退 3: scatter_add 全 scale 0.20x–0.26x

- **根因**：fp32 working buffer 引入额外 .clone().to(fp32).contiguous() 开销；atomic RMW 在高竞争下串行化
- **已知限制**：ntops13/自测结果_scatter_add_with_skill.md 中标注"性能未达标"；属 atomic contention + wrapper overhead 双重上限

回退 4: pixel_unshuffle 小 shape 0.26–0.29x

- **根因**：MetaX GPU kernel launch 最低延迟 ~0.04ms，<100K elements 的输入下该固定开销主导整体耗时

- **诊断:** host overhead 仅 0–6% (非 Python dispatch 问题), 瓶颈在 GPU kernel 本身
- **block_size sweep:** 128 → 256 → 512 → 1024 → 2048 → 4096 → 8192, 最优区间 1024–4096 (0.84–0.85x), 选定 4096 作为默认
- **大 shape 反超:** (8,64,256,256) (33M elements) 上 ntops **反超 torch**: 1.08x fp32 / 1.09x fp16, 带宽利用率 144.9 vs 134.3 GB/s
- **结论:** 小 shape 属 launch overhead 框架上限 (无法通过 tile 消除), 大 shape 达标且反超

5. 失败诊断案例、修复过程或规避建议

5.1 案例 A: copysign 的 IEEE 754 ±0.0 语义陷阱

- **症状:** `nt1.where(other >= 0, abs(input), -abs(input))` 在 `other = -0.0` 时给出错误符号
- **根因:** IEEE 754 中 `-0.0 >= 0.0` 为 True, `< 0.0` 无法识别 `-0.0`
- **Skill 预防:** `API_REFERENCE.md` \$ `libdevice.copysign` 明确将该模式列为反例
- **A 版 (Skill 辅助):** 直接绕过该坑, 1 轮成稿
- **B 版 (无 Skill):** Agent 自我否定 2 次 (V1→V2→V3), 浪费 ~200 行对话上下文

5.2 案例 B: addcmul fp16 精度 bug (FC-15)

- **症状:** `test_addcmul_large FAIL`, `max_diff ≈ 0.0312` (`value=3, 4096²`)
- **根因:** `prod = (t1.fp32 * t2.fp32).to(fp16)` 后, `input + v * prod` 在 fp16 执行, 误差随 `|value|` 线性放大
- **修复:** 4 个 operand 全部 upcast 到 fp32, 整段表达式在 fp32 计算, 最后 cast 回 `input.dtype`
- **验证:** fp16 MARE 从 `9.7e-3 ~ 1.45e-2` (FAIL) → **0.0 (bitwise exact)**

5.3 案例 C: nansum Triton 编译失败 (FC-17 类)

- **症状:** R1 编译时 `dtype.dtype` attribute path 错误
- **根因:** FC-17 (`dtype` attribute path): `output = nt1.sum(...).to(input.dtype)` 触发生成 IR 中非法的 cast 链
- **修复:** 删除末尾 cast, `output[0] = nt1.sum(block, axis=0)` 让 store 自动 cast 回输出 dtype
- **修复轮数:** 1 轮 (Agent 根据生成 IR 反推 fix)

5.4 案例 D: scatter_add fp16 全 FAIL (FC-15 延伸)

- **症状:** R1 V1 kernel 在 fp16 上 `max_diff 1.2e+01 ~ 3.3`
- **根因:** `tl.atomic_add` 在 fp16 指针上产生非确定累加误差
- **修复:** 引入 fp32 working buffer, kernel 总在 fp32 累加, 最后 `out.copy_(work.to(dtype))`
- **验证:** 34/65 → **65/65**, fp16 全 PASS

5.5 案例 E: 远端 editable install 指向错位

- **症状:** `import ntops.kernels.nansum` 在 ntops5 报 `ModuleNotFoundError`
- **根因:** 远端 `sys.path` 把 `/data/ntops6/src` 排在前面
- **修复:** 把 `kernel/wrapper/_init__.py` 增量同步到 ntops6, 清 `__pycache__`
- **规避建议:** 在 Skill 中强调“活跃包确认”作为 Step 0 必检项

5.6 案例 F: pixel_unshuffle 绕过九齿 (Step 0.8 预防 → 实测验证有效)

- **历史症状** (`scatter_add` 事故外推): `layout/view` 族算子容易被 Agent 把 `view-only` 主操作包装成“1D flatten + 索引计算”后声称是九齿实现
- **预防机制:** 在 `SKILL.md` 中加入 Step 0.8 ( IMPLEMENTATION PROPOSAL), 任何算子动笔前必须先输出 5 项简报 (分类 / 方案 / 是否 GPU kernel / 风险 / 备选), 第 3 项强制声明“主操作是否 view-only”
- **实测验证** (`ntops14/pixel_unshuffle`): Agent 严格走完 Step 0.8, 输出完整 5 项简报并等用户选 Plan A 后才动笔; kernel 用的是真正的 Pattern 13 九齿 1D copy (`output = source`), wrapper 用 `view + permute` 处理索引逻辑; 全程**没有绕过九齿、没有 silent fallback**
- **对照:** `scatter_add` 事故 (旧 Skill, 无 Step 0.8) → `pixel_unshuffle` (新 Skill, Step 0.8 强制) → 防护有效

5.7 案例 G: pixel_unshuffle permute 维度顺序错误 (代码级 bug)

- **症状:** R2 测试 shape 匹配但数值错, `max_diff = 4.08577299118042`
- **根因:** Agent 推导 `pixel_unshuffle` 的 `reshape+permute` 索引映射时出错
 - 4D 错: `view(N,C,r,r,H,W).permute(0,1,4,2,5,3)`

- 3D 错: `view(C,r,r,H,W).permute(0,3,1,4,2)`
- 修复: 用 `(1,1,4,4)` 小 tensor 做逐元素 trace, 对比 `F.pixel_unshuffle` 真值, 反推正确映射:
 - 4D 对: `view(N,C,H,r,W,r).permute(0,1,3,5,2,4)`
 - 3D 对: `view(C,H,r,W,r).permute(0,2,4,1,3)`
- 验证: R3 quick test `match=True, max_diff=0.0`; 完整 `pytest 12/12 PASS`
- FC 卡片: 未触发 (代码级 bug, 非九齿语法 / 平台 API / 精度陷阱)
- 规避建议: 对 `layout/view` 族算子的 `permute` 索引推导, 应在 Step 2 之前先用 `numpy / torch CPU` 做小规模逐元素验证

6. 与不使用 .skill 的 AI 智能体基线对比

6.1 A/B 实验设计

- A 版: 使用 `nt-devskill` 实现 `copysign` (测试场地/`ntops11`)
- B 版: 不激活 Skill, Agent 仅凭仓库中已有算子自学 (测试场地/`ntops12`)
- 输入提示词完全相同
- 对照汇总: 测试场地/`copysign_AB_汇总.md`

6.2 关键指标对照

维度	A. Skill 辅助	B. 无 Skill
首次上传到 GPU 即 PASS	✔ 12/12 in 14.19 s	✗ 未上传 (仅本地 parse)
实测正确性 (MetaX C500)	✔ fp32/fp16 + ±0.0 + nan/inf	✗ 无任何 GPU 验证
Benchmark 数据	✔ 6 组 shape×dtype	✗ 无
性能调优动作	tile sweep 6 档, block 1024→4096	无
代码迭代轮数	1 轮 (一次成稿)	3 轮 (V1→V2→V3 两次推翻)
测试用例数	12	10 (缺 signed-zero 专项)
Wrapper 边界检查	shape + dtype 双 assert	无
IEEE 754 ±0.0 陷阱	Skill 文档明确禁止 < 0.0	Agent 自悟踩坑后自纠
MCP 远程测试	✔ 完成闭环	✗ 未调用

6.3 对照结论

维度	结论
完成度	A 版端到端闭环, B 版停在"写完未测"
正确性信心	A 版实测 12/12 + IEEE 754 专项, B 版仅纸面分析
性能信心	A 版有 6 档 tile sweep + 最终 benchmark; B 版无数据
迭代成本	A 版 1 轮成稿, B 版 3 轮 (2 次自我推翻)
关键坑规避	Skill 文档把 <code>nt1.where(< 0.0, ...)</code> 列为反例, 直接绕过

一句话: Skill 的主要价值不是"写代码更快", 而是把已知的坑提前告诉 Agent, 避免无谓的迭代与未验证就交付。

7. 安全、依赖、授权与引用披露

7.1 安全

- 不含密钥、凭证、私有 token、未授权数据

- 不指示 Agent 删除测试、伪造 benchmark、绕过验证或隐藏失败
- 远程 MCP/SSH 上传遵循 `__init__.py` 增量修改协议，避免覆盖用户未提交改动
- MCP 安全机制**（详见 § 7.6）：
 - `config.json` 含 SSH 密码或密钥路径，必须列入 `.gitignore`，本次提交仅含 `config.example.json` 模板
 - `remote_rm` 内置危险路径保护：拒绝删 `remote_root / /data / /opt` 等，删目录需显式 `recursive=True`
 - 所有命令通过 SSH session 在远端运行，与 MCP 客户端进程隔离；不会在本地执行任何远端返回的 payload

7.2 依赖

依赖	用途	公开来源
ninetoothed	DSL 编译器	https://github.com/InfiniTensor/ninetoothed
triton	后端 IR	https://triton-lang.org/
torch	参考实现、CUDA timing	https://pytorch.org/
MetaX GPU 驱动	C500 硬件	vendor-provided
mcp>=1.27.2（MCP 依赖）	MCP Server SDK（stdio 协议）	https://github.com/modelcontextprotocol/python-sdk
paramiko>=5.0.0（MCP 依赖）	SSH/SFTP 远程连接	https://www.paramiko.org/
uv（MCP 构建）	Python 包管理与虚拟环境	https://docs.astral.sh/uv/

7.3 授权

- 本 Skill 包中的指令文本、诊断脚本、故障卡片、模板均为原创
- `examples/` 目录下示例算子参考 NineToothed 官方 examples（<https://github.com/InfiniTensor/ninetoothed-examples>），未 vendor 源码
- 未 vendor 任何第三方竞赛仓库代码
- `remote-operator-mcp`：MCP Server 实现为原创，仅调用 `mcp / paramiko` 两个公开依赖，未 vendor 任何第三方代码

7.4 AI 辅助披露

本 Skill 包在开发过程中使用 AI 辅助：

- 组织工作流指令与决策树
- 创建可复用代码模板与诊断脚本
- 准备可复现性验证脚本

最终的正确性结果、benchmark 结果、环境细节与参与者身份均由参与者本人在提交前核实。

7.5 引用

详见 `REFERENCE.md`：

- NineToothed 官方仓库
- NineToothed examples 仓库
- NineToothed 文档
- PyTorch 文档
- Triton 文档
- MetaX GPU 文档
- 竞赛官方材料与参考 Skill
- MCP 协议规范（Model Context Protocol）

7.6 配套 MCP： remote-operator-mcp

为了让 `Step 0 → Step 3` 的闭环能在远程 MetaX GPU 上实跑，本次提交在 `remote-operator-mcp/` 目录下附带一个独立 MCP Server。**Skill 与 MCP 解耦**：Skill 不依赖任何特定 MCP 实现；`SKILL.md` 中所有“远程 GPU”路径仅以本 MCP 的工具命名为示例，用户可替换为其他兼容 MCP 实现。

7.6.1 12 个 MCP 工具

类别	工具
连接与传输	ping · upload_code (目录自动 tar.gz) · download_file
测试与运行	run_test (自动注入 MetaX 环境变量 + tee 日志) · run_python (base64 传输) · auto_bench (runs>1 聚合 median/min/max)
远端文件操作	remote_ls · remote_cat · remote_md5 · remote_glob (支持 **) · remote_rm (含危险路径保护)
GPU 监控	mx_smi (支持 -l 1000 / --show-usage / -i <idx> 等参数)

7.6.2 安装到常用 Agent (stdio MCP 配置)

所有 Agent 均为 stdio 模式； --directory 必须填 MCP 目录的绝对路径； config.json 必须在 MCP 目录下且含有效 SSH 凭据。

Agent	配置文件	配置片段
Qoder (qoderclcn)	~/.qoder-cn/mcp.json (用户级) 或 .qoder/mcp.json (项目级)	{ "mcpServers": { "remote-operator": { "command": "uv", "args": ["--directory", "/abs/path/to/remote-op
Cursor	Cursor Settings → MCP → + Add new MCP server	Type command ; Name remote-operator ; Command uv --directory /abs/path/to/remote-operator
Claude Code (Anthropic)	~/.claude.json 或项目 .mcp.json	同 Qoder
Windsurf (Cascade)	Windsurf Settings → MCP → Edit Config	同上 stdio 配置
Cline / Roo Code	VSCode settings.json → Cline MCP Settings	{ "mcpServers": { "remote-operator": { "command": "uv", "args": ["--directory", "/abs/path/to/remote-op
OpenAI Codex CLI	~/.codex/config.toml 或项目 codex.toml	[mcp_servers.remote-operator] command = "uv" args = ["--directory", "/abs/path/to/remote-oper
GitHub Copilot Workspace	Workspace MCP 面板	同上 command / args

安装步骤 (以 Qoder 为例)：

```
cd /abs/path/to/remote-operator-mcp
uv sync                                # 安装 mcp + paramiko
cp config.example.json config.json     # 填写 host/user/password 或 key_filename/remote_root
# 在 ~/.qoder-cn/mcp.json 中追加上述 stdio server 配置 → 重启 Qoder
```

7.6.3 协同验证

```
→ ping                                # 确认 MCP 在线
← pong
→ upload_code(src="./ntops7", dst="ntops7/") # 上传算子工程
→ run_test(cmd="python tests/test_nansum.py", workdir="ntops7")
← 自动 tee 到 logs/test_nansum_<timestamp>.log
→ auto_bench(v0="fallback.py", v1="MateX.py", workdir="05_diag", runs=3)
← 输出 PASS/FAIL + speedup + median/min/max
```

7.6.4 Skill ↔ MCP 的关系

- SKILL.md 在 Step 0 通过检查 run_test / upload_code / run_python 等工具是否可用来选择"本地 GPU / MCP 远程 / SSH"执行策略
- 所有 5 个自测任务 (copysign / nansum / addcmul / scatter_add / pixel_unshuffle) 与 A/B 对照均通过本 MCP 在 MetaX C500 远端完成
- 用户若使用其他 MCP 实现，只需保证暴露同名工具 (upload_code / run_test / run_python 为最低要求)，Skill 即可正常工作

8. 后续可维护计划

8.1 短期（本次提交后 1 个月）

- **补充 layout/view 族示例**: pixel_unshuffle 已在自测中验证（ntops14/），下一步把 Pattern 13 + wrapper-heavy 路径沉淀到 examples/pixel_unshuffle/ 与 examples/channel_shuffle/，作为 layout/view 族的官方示范
- **扩展 FIX_CARDS 至 30 张**: 当前 25 张，计划补充 MetaX 特有的 atomic RMW 实现差异、AOT build 配置失败模式、layout/view 族 permute 索引推导错误（来自 pixel_unshuffle R2 的 Case G）
- **增加 scripts/check_submission.py**：一键验证提交完整性（HONOR_CODE/REFERENCE/README/PR 模板/5 个自测报告）
- **MCP: 补充单元测试**: 为 remote-operator-mcp 增加本地 mock-SSH 的 pytest 套件，覆盖 12 个工具的 happy path 与错误分支（连接失败、危险路径拦截、tar.gz 打包边界）
- **MCP: 目录递归上传**: 当前 upload_code 只处理普通文件，计划支持递归子目录与符号链接

8.2 中期（3 个月）

- **跨平台测试**: 当前仅验证 MetaX C500 + CUDA；扩展到 AMD ROCm、Intel XPU
- **CI 集成**: 把 scripts/pipeline.py 接入 GitHub Actions，每次 PR 自动跑 5 个自测算子的 correctness
- **A/B 实验扩展**: 除 copysign 外，对 nansum / addcmul / scatter_add 各做一组无 Skill 对照
- **MCP: 多凭据后端**: 支持 SSH agent forwarding、GSSAPI/Kerberos、Jump Host 跳板
- **MCP: SSE/HTTP 传输**: 在 stdio 之外提供 HTTP+SSE 传输模式，便于 Web IDE（如 GitHub Codespaces）接入

8.3 长期（6 个月+）

- **NineToothed 版本升级跟进**: 当前针对 ninetoothed 0.x；后续跟进 1.x 的新语法（如 shared memory 抽象）
- **自动 proposal 生成**: 把 Step 0.8 的  IMPLEMENTATION PROPOSAL 模板进一步结构化，让 Agent 能以 JSON schema 输出
- **Skill 版本管理**: 把 FIX_CARDS 与 CODE_TEMPLATES 拆成独立子包，允许用户按需扩展
- **MCP: 多集群调度**: 支持一次配置多个远端 GPU 节点，按任务特征（dtype / shape / 卡号）路由

附录 A：命令与日志路径汇总

算子	命令	日志
copysign	pytest tests/test_copysign.py -v	/data/logs/test_copysign_*.log
nansum	python tests/test_nansum.py	/data/logs/test_nansum_20260712_042759.log
addcmul	pytest tests/test_addcmul.py -v	/data/logs/test_addcmul_20260712_121518.log
scatter_add	pytest tests/test_scatter_add_correctness.py	/data/logs/bench_scatter_add_20260712_142744.log
pixel_unshuffle	pytest tests/test_pixel_unshuffle.py -v	/data/logs/test_pixel_unshuffle_*.log
copysign benchmark	python bench_copysign.py	/data/logs/bench_copysign_*.log
addcmul benchmark	python scripts/bench_addcmul.py	/data/logs/bench_addcmul_20260712_121446.log
scatter_add inspect	python tests/inspect_scatter_add.py	/data/logs/inspect_scatter_add_20260712_142728.log
pixel_unshuffle benchmark	python tests/bench_pixel_unshuffle.py	/data/logs/bench_pixel_unshuffle_*.log

附录 B：环境

- **GPU**: MetaX C500
- **OS**: Linux (远程 MCP)
- **Python**: 3.10
- **torch**: 2.6.0+metax3.2.1.3
- **ninetoothed**: 最新版（vendor-provided）
- **triton**: vendor-provided MetaX fork

附录 C：合规清单

- ✓ HONOR_CODE.md 已签署
- ✓ REFERENCE.md 已披露
- ✓ README.md 已说明安装与使用
- ✓ PR_DESCRIPTION_TEMPLATE.md 已填充
- ✓ 5 个自测报告已按 自测结果模版.md 完成 (覆盖 elementwise / reduction / composition / scatter / layout-view 5 族)
- ✓ 所有命令可复现
- ✓ 所有失败案例已披露 (含 scatter_add 绕过九齿事故 + pixel_unshuffle Step 0.8 正面验证)
- ✓ 无密钥、凭证、隐藏答案、针对性 bypass
- ✓ 配套 MCP (remote-operator-mcp/) 已交付: 含 README.md (12 个工具说明) + pyproject.toml + uv.lock + remote_operator_mcp.py + config.example.json
- ✓ 多 Agent MCP 安装指引已覆盖: Qoder / Cursor / Claude Code / Windsurf / Cline / Codex CLI / GitHub Copilot Workspace
- ✓ MCP 凭据安全: config.json 未提交 (已 .gitignore) ; config.example.json 仅含占位符
- ✓ MCP 安全防护: remote_rm 含危险路径保护 (拒绝删 remote_root / /data / /opt , 删目录需显式 recursive=True)